

After setting up the kafka in wsl , our next move is to setup the redis and postgresql with it.

Requirements Docker with wsl 2 integration setup instead of Hyper-V.

The recommended approach is to install Docker Desktop on Windows and enable WSL 2 integration:

1. Install Docker Desktop on Windows (if not already installed):
 - Download from: <https://www.docker.com/products/docker-desktop/>
 - During installation, ensure "Use WSL 2 instead of Hyper-V" is selected
2. Enable WSL 2 Integration for Kali:
 - Open Docker Desktop
 - Go to Settings → Resources → WSL Integration
 - Toggle on Kali-linux (your distribution name)
 - Click Apply & Restart

3. Verify Docker Works in Kali:

```
docker --version
docker run hello-world
```

Setting Up Redis and PostgreSQL in WSL

The fastest way to get these dependencies running is via Docker inside your WSL environment.

Run these two commands in your WSL terminal to spin them up instantly:

Spin up Redis

```
docker run --name aurastream-redis -p 6379:6379 -d redis:alpine
```

```
mrldiot@AMIT: ~  
$ docker run --name aurastream-redis -p 6379:6379 -d redis:alpine  
Unable to find image 'redis:alpine' locally  
alpine: Pulling from library/redis  
05922fb64b47: Pull complete  
eeab38c295a8: Pull complete  
dc3229b74aab: Pull complete  
4f4fb700ef54: Pull complete  
5da8100e7e68: Pull complete  
e6f31ffc071e: Pull complete  
181cd4ffd5c9: Pull complete  
Digest: sha256:9d317178ecec8454a2284a9e6df2466b93c745529947f0cd42a0fa9609d7005  
Status: Downloaded newer image for redis:alpine  
41ec165e3461915c88f9283adbd218f95fcc2d86aba8a8be70635cac5614ad29
```

Spin up PostgreSQL

```
docker run -d --name aurastream-postgres \  
-e POSTGRES_PASSWORD=securepassword123 \  
-e POSTGRES_DB=aurastream \  
-p 5432:5432 \  
postgres:alpine
```

```
(mrIdiot@AMIT)~  
$ docker run --name aurastream-postgres -e POSTGRES_PASSWORD=mysecretpassword -e POSTGRES_DB=aurastream -p 5432:5432 -  
d postgres:alpine  
Unable to find image 'postgres:alpine' locally  
alpine: Pulling from library/postgres  
21225587b6ee: Pull complete  
c24407d2ec97: Pull complete  
d5392b8b2486: Pull complete  
d5c3772878da: Pull complete  
55afa1ecc21d: Pull complete  
c789cfcee1a8: Pull complete  
9530417d1dbb: Pull complete  
8c4cf0b70797: Pull complete  
76b8b44efa5a: Pull complete  
Digest: sha256:9a8afca54e7861fd90fab5fd4c42477a6b1cb7d293595148e674e0a3181de15  
Status: Downloaded newer image for postgres:alpine  
277bec518224aad8f4eb7da5e8d1d2cc7b869c721d9f1e90a85fca01f00f6a00
```

Modifying Your Go Consumer Code

You will update your **Processing Service (Consumer)** to handle the high-throughput logic. For every financial event coming out of Kafka, your Go code will execute the **Write-Through/Cache-Aside pattern**:

1. Parse the incoming JSON message from Kafka into a Go `struct`.

2. Write the transaction data directly to **PostgreSQL** for permanent storage.
3. Cache the transaction or the user's updated balance in **Redis** with an expiration time (TTL).

Go Code Architecture for the Consumer

To implement this, you'll need the Go Redis driver

(github.com/redis/go-redis/v9)(<https://github.com/redis/go-redis/v9>))

and the PostgreSQL driver (github.com/lib/pq)(<https://github.com/lib/pq>)).

Write the Next Modified code for consumer script with above changes.

Create the Database Schema (PostgreSQL)

Connect to your running `aurastream-postgres` container and create the table.

Run this command in your Kali terminal:

1. Open the PostgreSQL shell

```
docker exec -it aurastream-postgres psql -U postgres -d aurastream
```

Command to create the table :

```
CREATE TABLE IF NOT EXISTS transactions (  
    id SERIAL PRIMARY KEY,  
    transaction_id VARCHAR(255) UNIQUE NOT NULL,  
    user_id VARCHAR(255) NOT NULL,  
    amount NUMERIC(19, 4) NOT NULL,  
    currency VARCHAR(3) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. List all databases

```
\l
```

3. Show the current database

```
SELECT current_database();
```

4. List all tables

```
\dt
```

5. Describe a table (show columns)

```
\d transactions
```

or

```
\d+ transactions
```

6. View all data in the **transactions** table

```
SELECT * FROM transactions;
```

7. View only the first 10 rows

```
SELECT * FROM transactions LIMIT 10;
```

8. Count the number of rows

```
SELECT COUNT(*) FROM transactions;
```

9. Exit PostgreSQL

```
\q
```

Initialize Go Modules and Install Dependencies

In your project directory (where `main.go` is), run:

```
# Initialize module
go mod init aurastream

# Download dependencies
go get github.com/lib/pq
go get github.com/redis/go-redis/v9
```

```
go get github.com/segmentio/kafka-go
```

```
D:\Projects\go_lang\redis-postgresql-kafka>go mod init aurastream
go: creating new go.mod: module aurastream

D:\Projects\go_lang\redis-postgresql-kafka>go get github.com/lib/pq
go: downloading github.com/lib/pq v1.12.3
go: added github.com/lib/pq v1.12.3

D:\Projects\go_lang\redis-postgresql-kafka>go get github.com/redis/go-redis/v9
go: downloading github.com/redis/go-redis v6.15.9+incompatible
go: downloading github.com/redis/go-redis/v9 v9.21.0
go: downloading go.uber.org/atomic v1.11.0
go: downloading github.com/cespare/xxhash/v2 v2.3.0
go: added github.com/cespare/xxhash/v2 v2.3.0
go: added github.com/redis/go-redis/v9 v9.21.0
go: added go.uber.org/atomic v1.11.0

D:\Projects\go_lang\redis-postgresql-kafka>go get github.com/segmentio/kafka-go
go: added github.com/klauspost/compress v1.15.9
go: added github.com/pierrec/lz4/v4 v4.1.15
go: added github.com/segmentio/kafka-go v0.4.51
```

Try to run the go script using command in cmd : go run main.go

After that you need to run your go script and if you're getting any error related to password authentication failed for postgres then it may be happening because of you already have installed postgres in windows machine so you need to shut its process first

Open powershell with admistration:

Run

```
Stop-Service -Name "postgresql*" -Force
netstat -ano | findstr 5432
```

Then you should see only one process on 5432 port and then run your go script again.

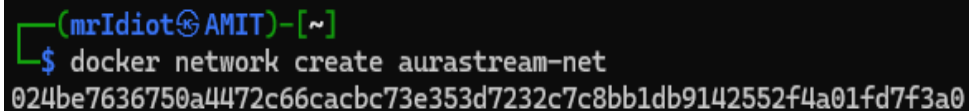
Produce a Test Message to Kafka

Before running your Go app, you need data in Kafka. Run this in your terminal to send a test message:

Open 2 wsl terminals

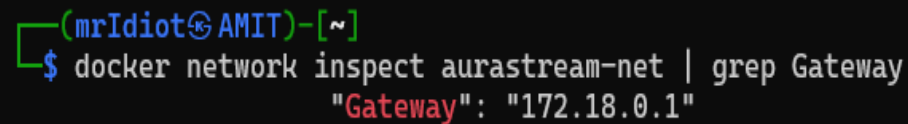
In Terminal 1:

```
docker network create aurastream-net
```



```
(mrIdiot@AMIT)-[~]  
$ docker network create aurastream-net  
024be7636750a4472c66cacbc73e353d7232c7c8bb1db9142552f4a01fd7f3a0
```

Check your port : `docker network inspect aurastream-net | grep Gateway`



```
(mrIdiot@AMIT)-[~]  
$ docker network inspect aurastream-net | grep Gateway  
"Gateway": "172.18.0.1"
```

Use the same above ip in below command and run the command:

```
docker run -d --name aurastream-kafka \ --network=aurastream-net \ -p 9092:9092 \  
-e KAFKA_BROKER_ID=1 \ -e KAFKA_ZOOKEEPER_CONNECT=aurastream-zookeeper:2181 \ -e  
KAFKA_LISTENERS=INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092 \ -e  
KAFKA_ADVERTISED_LISTENERS=INTERNAL://aurastream-kafka:29092,EXTERNAL://172.18.0.  
1:9092 \ -e  
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTE  
XT \ -e KAFKA_INTER_BROKER_LISTENER_NAME=INTERNAL \ -e  
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 \ confluentinc/cp-kafka:7.4.0
```

```

(mrIdiot@AMIT) ~/kafka_2.13-3.9.0
$ docker run -d --name aurastream-zookeeper --network=aurastream-net -e ZOOKEEPER_CLIENT_PORT=2181 -
e ZOOKEEPER_TICK_TIME=2000 confluentinc/cp-zookeeper:latest
Unable to find image 'confluentinc/cp-zookeeper:latest' locally
latest: Pulling from confluentinc/cp-zookeeper
8f10199ed94b: Pull complete
2e8a7df9c2ee: Pull complete
10f05dd8b1db: Pull complete
71a9f6a9ab4d: Pull complete
9c266ba63f51: Pull complete
da3ed5db7103: Pull complete
e444bcd4d577: Pull complete
f963a77d2726: Pull complete
79161a3f5362: Pull complete
41dac8b43ba6: Pull complete
f3a82e9f1761: Pull complete
45fd2fec8a19: Pull complete
c955f6e31a04: Pull complete
eabd8714fec9: Pull complete
eca0188f477e: Pull complete
Digest: sha256:7610a50b13e7e9f6dc0dd00585ffdc0bbed27392bef6b45791ef4087b2a746c
Status: Downloaded newer image for confluentinc/cp-zookeeper:latest
0acbdl62d3de22a82bfc6aa8e89565b591fd1261594ebf6de4f6eab18cd9f7ae

```

NOTE : IN ABOVE COMMAND WE ARE USING CONFIGURATIONS AND THOSE SHOULD MATCH WITH YOURS., SO IF YOU ARE GETTING ANY ERROR RELATED PORT,IP OR KAFKA VERSION YOU NEED TO MODIFY THIS COMMAND WITH YOUR CONFIGURATION.

TO MATCH THE CONFIGURATION , FIRST REMOVE OLD ONE :

Step 1: Stop and Remove the Current Kafka Container

```

bash
docker stop aurastream-kafka
docker rm aurastream-kafka

```

THEN CREATE NEW WITH YOUR CONFIGURATIONS , LIKE I HAVE DONE BELOW

```

(mrIdiot@AMIT)-[~/kafka_2.13-3.9.0]
$ docker run -d --name aurastream-kafka \
  --network=aurastream-net \
  -p 9092:9092 \
  -e KAFKA_BROKER_ID=1 \
  -e KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 \
  -e KAFKA_LISTENERS=PLAINTEXT://0.0.0.0:9092 \
  -e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \
  -e KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 \
  confluentinc/cp-kafka:7.4.0
504f6a966f43d55a96ef08ebfc41db865d3aab1c4a0736d1238690618ab6eee2

(mrIdiot@AMIT)-[~/kafka_2.13-3.9.0]
$ netstat -tlnp | grep 9092
(No info could be read for "-p": geteuid()=1000 but you should be root.)
tcp6      0      0 :::9092          :::*             LISTEN      -

(mrIdiot@AMIT)-[~/kafka_2.13-3.9.0]
$ ss -tlnp | grep 9092
LISTEN 0      4096             *:9092           *:*
```

THEN RUN IT WITH BELOW COMMANDS:

```

(mrIdiot@AMIT)-[~/kafka_2.13-3.9.0]
$ ip route | grep default
default via 172.21.64.1 dev eth0 proto kernel

(mrIdiot@AMIT)-[~/kafka_2.13-3.9.0]
$ # Replace 172.28.0.1 with the IP you found in Step 1
docker run --rm --network=aurastream-net confluentinc/cp-kafka:7.4.0 kafka-console-producer \
  --bootstrap-server 172.21.64.1:9092 \
  --topic financial-transactions
```

SEE IN ABOVE SCREENSHOT I WAS GETTING MY IP MISMATCH ISSUE SO IT CHECKED MY IP AND CHANGED THE IP IN MY COMMAND.

docker ps

HERE YOU WILL SEE FOUR CONTAINERS RUNNING

The smoking gun is right there in your `docker ps -a --filter name=zookeeper` output: **it's completely blank.**

You do not have a Zookeeper container running at all!

Start Zookeeper

Run this command to create and start a Zookeeper container named `aurastream-zookeeper` on your `aurastream-net` network:

Bash

```
docker run -d --name aurastream-zookeeper \
  --network=aurastream-net \
  -p 2181:2181 \
  -e ZOOKEEPER_CLIENT_PORT=2181 \
  -e ZOOKEEPER_TICK_TIME=2000 \
  confluentinc/cp-zookeeper:7.4.0
```

Step 2: Recreate and Start Kafka

Now that Zookeeper is alive, remove the broken Kafka container and spin up the dual-listener setup again:

Bash

```
docker rm -f aurastream-kafka

docker run -d --name aurastream-kafka \
  --network=aurastream-net \
  -p 9092:9092 \
  -e KAFKA_BROKER_ID=1 \
  -e KAFKA_ZOOKEEPER_CONNECT=aurastream-zookeeper:2181 \
  -e KAFKA_LISTENERS=INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092 \
  -e
KAFKA_ADVERTISED_LISTENERS=INTERNAL://aurastream-kafka:29092,EXTERNAL://localhost:
9092 \
  -e
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTE
XT \
  -e KAFKA_INTER_BROKER_LISTENER_NAME=INTERNAL \
  -e KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 \
  confluentinc/cp-kafka:7.4.0
```

Step 3: Verify They Are Both Alive

Wait 10 seconds for them to complete their handshake, then check your running containers:

Bash

```
docker ps
```

You should now proudly see **four** containers running perfectly: `aurastream-postgres`, `aurastream-redis`, `aurastream-zookeeper`, and `aurastream-kafka`.

Don't create the containers again and again if you have runned the above commands once and created those containers , run below commands to start the containers if they got exited.

```
docker start aurastream-postgres, aurastream-redis,  
aurastream-zookeeper, aurastream-kafka
```

Step 4: Fire Your Message!

Everything is lined up now. Open up the console producer using the internal bridge address:

```
Bash
```

```
sudo apt install kafkacat -y
```

```
kcat -P -b localhost:9092 -t financial-transactions -X queue.buffering.max.ms=1
```

```
Once the prompt opens, press ctrl+d one time, drop your message string and hit  
Enter:
```

```
JSON
```

```
{"transaction_id": "tx-amit-010", "amount": 450.50, "currency": "INR", "user_id": "user-50"}
```

Keep your eyes on your Go app terminal window—it will catch the data processing stream instantly. **To exit from this command press ctrl+d again.**